

AEO Architecture Comparison — Executive Technical Review

Two independent implementations of the same `aeo_architecture_v4.md` specification, evaluated for a leadership decision.

- **Architecture A** — Kenneth's `page_crawler` (local working tree). Deterministic-first; custom async orchestrator; PostgreSQL as store and job queue.
- **Architecture B** — Sanjith's `aeo-pipeline` (github.com/Sanjith72/aeo-pipeline). LLM-first (Ollama/phi3); async-native; agent-per-block.

Prepared as a board-ready record. Every conclusion is grounded in direct code inspection; file and line citations are given throughout. Where evidence is weak or a claim could not be fully verified, it is flagged.

0. Executive snapshot (read this first)

Recommendation: adopt Architecture A as the production system of record, and port five specific, low-risk components from Architecture B. Do not merge the two codebases, and do not rebuild on B.

Both teams independently built the **same** v4 spec (both repositories ship the identical 276-line `aeo_architecture_v4.md`). They diverged on philosophy: **A is deterministic-first** (10 HTML-signal scorers; the LLM only refines 2 of them), **B is LLM-first** (Ollama drives blueprint, coverage, triage, recommendations, and audit).

	Architecture A (Kenneth)	Architecture B (Sanjith)
Source size	10,365 LOC / 107 files	2,400 LOC / 18 files
Tests	342 (335 unit + 7 integration), offline, in CI	1 smoke test; no <code>tests/</code> dir; no CI
Core philosophy	Deterministic-first, LLM-optional	LLM-first (every page hits Ollama)
v4 site-level coverage (missing pages)	Yes — ideal sitemap + thin-cluster diff	No — per-page coverage only
Flagship features wired into run path	Independent validator + adversarial auditor + retry loop all wired	Adversarial auditor + 4-track engine-routed evaluator built but not wired
Genuine standout strengths	Completeness, tests, scalability, cost, reproducibility	OpenTelemetry, per-domain YAML onboarding, dry-run demo, async-native ergonomics
20-dimension score (post-verification)	156 / 200	106 / 200

The margin is decisive and survives an adversarial fairness review that deliberately looked for "bigger-is-better" bias (see §8). A wins 17 dimensions, B wins 2 (Pipeline Complexity, Developer Experience), 1 tie (Observability).

How this review was conducted (methodology and evidence quality)

This is not a documentation read-through. The following was done:

- 1 **Both codebases were read in full at source level.** B (2,400 LOC) was read file-by-file first-hand. A (10,365 LOC) was mapped by seven parallel deep-reader agents, each auditing one subsystem and returning file/line-cited findings, plus a first-hand read of its orchestrator and design docs.
- 2 **B was cloned** from GitHub to a scratch directory (kept out of A's working tree) so it could be inspected at the same depth as A.

- 3 **A 20-dimension scoring** was produced by five independent scoring agents (4 dimensions each), each instructed to judge **fitness for purpose**, reward leanness, and **penalize features that are present-but-not-wired or claimed-but-untested**.
- 4 **Adversarial verification**. Two independent agents re-checked the load-bearing claims against the actual code and audited the scoring for bias toward the larger codebase. All six core claims were **confirmed**; the fairness audit produced four defensible upward adjustments for B, which have been applied (see §8).

Evidence-quality caveats, stated up front:

- A's documentation **overstates its own test count** ("346" / "332" / "240" appear in different docs); the **verified** count is **342** (335 unit + 7 integration). Reported here as 342.
- A's **integration tests are DB-gated and skip in CI**, so migrations 0001–0009 and real SQL round-trips are not exercised in continuous integration; live Gemini/Perplexity/PageSpeed paths run behind fakes. They are unit-tested with mocks but not proven against live services here.
- B's `pyproject.toml` declares `pytest`, `mypy --strict`, and `testpaths = ["tests"]`, but **no `tests/` directory exists** — the entire suite is one 223-line `smoke_test.py`. This is a claimed-but-absent test story.
- Neither system has been observed running against live infrastructure in this review (no DB, Ollama, Gemini, or Perplexity credentials present). Behavioral claims are from code, not execution.

PHASE 1 — Architecture Discovery

1.1 Architecture A — overview

A treats AEO as a **measurement-and-remediation pipeline** with the LLM kept on a tight leash. The defining decision: **eight of ten scoring criteria require zero LLM calls**, and the two that use one (`content_depth`, `stats_in_html`) degrade gracefully to deterministic when the model is unavailable (`scoring/scorers/__init__.py:36-47`, `nlp/llm.py:147-152`). This makes scores reproducible, cheap, and fast.

A also fully implements the v4 "content-strategy engine" expansion: a **versioned Reference Architecture Generator** (L1 competitor patterns + L2 curated framework + optional L3 Gemini synthesis) that produces an **ideal sitemap**, and a **site-level Coverage Diff** that answers "which pages are missing entirely" — not just "how good is this page" (`processor/coverage_diff.py`, `reference/blueprint.py`, `pipeline/reference_arch.py:100-125`).

Component breakdown (A)

Layer	Modules	Role
Crawler	<code>crawl/discovery.py</code> , <code>prioritize.py</code> , <code>fingerprint.py</code> , <code>politeness.py</code> , <code>retry.py</code> , <code>runner.py</code> , <code>transport.py</code> , <code>client.py</code>	Sitemap+recursive discovery, top-N prioritization, SHA-256 change gate, robots.txt + per-host rate limiting, Crawl4AI/Playwright fetch, force-IPv4 transport
Extractors	<code>extract/*</code> (12 pure functions)	meta, schema JSON-LD, Q&A, stats, entities, headings, links, readability, render-mode, EE-A-T, glossary, chunker — each pure (<code>html</code> , <code>soup</code> , <code>url</code>) -> <code>dict</code>
Scoring	<code>scoring/rubric.py</code> , <code>result.py</code> , <code>aggregator.py</code> , <code>scorers/*</code> (10)	10-criterion rubric as YAML; per-criterion failure isolation; opt-in ThreadPool parallelism with byte-identical output
Reference	<code>reference/blueprint.py</code> , <code>generator.py</code> , <code>framework.py</code> , <code>competitor_patterns.py</code> , <code>query_intent.py</code> , <code>loader.py</code> , <code>feedback.py</code>	Versioned blueprint (content-hash), closed-vocabulary Pydantic guardrails, validated-wins feedback
Gap analysis	<code>processor/gap_analysis.py</code> , <code>coverage_diff.py</code>	Dual-layer per-page gap (60% blueprint+rubric / 40% competitor) + site-level missing/thin-cluster diff
Recommender	<code>recommender/schema.py</code> , <code>entity.py</code> , <code>content.py</code> , <code>models.py</code>	Deterministic schema generation + grounded content/entity advisories

Layer	Modules	Role
Validation	<code>validation/validator.py</code> , <code>independent.py</code> , <code>adversarial.py</code> , <code>simulate.py</code>	Edit-efficacy re-score + retry ≤ 3 ; non-circular independent validator (TLDR/H1/JSON-LD + Perplexity); opt-in adversarial auditor
Reporting	<code>report/builder.py</code> , <code>render.py</code> , <code>site_builder.py</code>	Per-page report + site-level rollup pinned to blueprint version
Pipeline	<code>pipeline/orchestrator.py</code> , <code>reference_arch.py</code> , <code>stages.py</code> , <code>worker.py</code> , <code>analysis.py</code>	Custom async orchestrator; queue worker (FOR UPDATE SKIP LOCKED)
Storage	<code>storage/db.py</code> , <code>migrate.py</code> , <code>models.py</code> , <code>repos/*</code> (15), <code>migrations/*</code> (9)	psycopg2 sync pool; incremental versioned migrations; Postgres as store + queue
Observability	<code>obs/tracing.py</code> , <code>error_sink.py</code>	<code>agent_traces</code> table + <code>aeo trace</code> ; per-page Error Sink isolation

End-to-end flow (A)

`audit-cycle DOMAIN` chains: **discover** → **prioritize (top-N)** → **generate+pin blueprint** → **site-level Coverage Diff** → **crawl (Crawl4AI)** → **content-hash gate** → **extract (12)** → **score (10)** → [per scored page] **Dual-Layer Gap** → **recommend** → **re-score+retry ≤ 3** → **independent validator (+Perplexity)** → **adversarial auditor** → **per-page report** → **site report** → **validated-wins feedback (human-gated)**. Each page is isolated by the Error Sink; the analysis tier fans out across a thread pool. See `architecture_diagrams/01_architecture_A_current.md`.

Technology stack (A)

Python ≥ 3.11 · Typer CLI · **psycopg2 (sync)** + **PostgreSQL 16** (store + queue) · Crawl4AI/Playwright (page fetch) · httpx (discovery, PageSpeed, Perplexity, LLM) · BeautifulSoup · PageSpeed Insights API · Ollama or cloud LLM (optional) · Perplexity API (optional) · structlog · pydantic-settings · YAML config · Docker + docker-compose (db/migrate/worker/app/ollama) · GitHub Actions CI (ruff + pytest → GHCR) · systemd timer / cron · OCI Ampere ARM target.

1.2 Architecture B — overview

B treats AEO as an **AI-native reasoning pipeline**. The defining decision is the mirror image of A's: **the LLM is the engine, not a refiner**. Ollama/phi3 generates the blueprint, judges semantic query coverage, triages pages, writes recommendations, and (in unwired code) audits them (`config.py:19` comment: "all LLM calls"). Determinism is reserved for entity/schema/citation/freshness checks and the SHA-256 change gate.

B is **async-native end to end** (asyncpg pool, httpx async, `asyncio.gather` fan-outs), markedly leaner, and ships several operational niceties A lacks: real **OpenTelemetry** tracing, **per-domain YAML onboarding**, a ``--dry-run`` **in-memory demo mode**, a **uniform force-IPv4** client factory, and a first-class **engine-target** enum (Perplexity/ChatGPT-Search/Gemini/generic).

Component breakdown (B)

Layer	Modules	Role
Agents	<code>agents/crawler.py</code> , <code>coverage_diff.py</code> , <code>processor.py</code> , <code>recommender.py</code> , <code>reference_generator.py</code> , <code>validator.py</code>	One file per block; crawler is no-LLM, the rest are Ollama-driven
Models	<code>models/blueprint.py</code>	Pydantic v2: frozen blueprint core, 30-day lock, content-hash, coverage/recommendation/validation/audit models
Storage	<code>db/schema.sql</code> , <code>db/queries.py</code>	Single idempotent <code>schema.sql</code> ; asyncpg; UUID PKs (pgcrypto)
Config	<code>config.py</code> , <code>domain_config.py</code> , <code>domains/*.yaml</code>	pydantic-settings + per-domain YAML (rapid7.com fully fleshed: 10 seed queries, 17 entities, 11 competitors)
Utils	<code>utils/http.py</code> , <code>utils/observability.py</code>	Uniform IPv4 client factory; OpenTelemetry OTLP + structlog

Layer	Modules	Role
CLI	<code>cli.py</code>	Typer + rich; <code>run</code> (with <code>--dry-run/--all/--regenerate-blueprint</code>), <code>db-init</code> , <code>report</code> , <code>blueprint</code>
Infra	<code>docker-compose.yml</code> , <code>systemd/*</code>	Postgres-only compose (schema as initdb); weekly systemd timer

End-to-end flow (B)

`run DOMAIN` chains: **load domain YAML** → **blueprint** (Ollama, or reuse if locked) → **crawl** (async BFS + hash gate) → **[processor] Ollama triage if >50** → **per-page coverage diff** via ``asyncio.gather`` (Ollama semantic query coverage + deterministic entity/schema) → **recommend** (Ollama, sequential loop) → **validate** (deterministic gates) → **persist** → ``aeo report`` table. A `--dry-run` variant runs the whole thing in memory, ≤10 pages, no DB. See [architecture_diagrams/02_architecture_B_current.md](#).

Technology stack (B)

Python ≥3.11 · Typer + rich CLI · **asyncpg (async) + PostgreSQL 16** · httpx async (http2) crawl + Ollama · BeautifulSoup + lxml · **Ollama/phi3 (all reasoning)** · `google-genai` (declared, reserved, unused) · **OpenTelemetry SDK + OTLP gRPC exporter** · structlog · pydantic v2 + pydantic-settings · tenacity · xxhash (declared, unused) · per-domain YAML · docker-compose (Postgres only) · systemd weekly timer · `mypy --strict` configured.

PHASE 2 — Deep Technical Comparison (20 dimensions)

Scores are 1–10, judged on **fitness for purpose**, not size. The four B scores marked with † were adjusted upward after the adversarial fairness review (§8); raw agent scores are shown in parentheses. No A score was inflated to compensate.

#	Dimension	A	B	Winner	One-line rationale
1	Modularity	8	7	A	A: pure-function scorer registry + seam-per-stage. B cleaner/coarser but 2 modules carry unwired public surface.
2	Extensibility	8	6	A	A: new criterion = YAML + scorer file; closed-vocab guardrails. B's best extension points (tracks, auditor) aren't reachable.
3	Pipeline complexity (lower better)	6	7	B	B is dramatically leaner and internally consistent — but partly because it ships less capability.
4	Agent architecture quality	8	5	A	A's engine-routing + auditor are wired and tested; B's are dead code (<code>smoke_test.py</code> only).
5	Scalability	8	5† (4)	A	A: Postgres queue + N workers. B: single-process, sequential domain loop, <code>url LIKE %domain%</code> multi-tenant bug.
6	Fault tolerance	8	5	A	A: layered isolation (<code>page_guard</code> , scorer floor, queue retry/dead-letter). B shallower; no resumable run.
7	Performance (throughput)	8	4	A	A extract-dominated (32–66 pages/s/core claimed); B model-bound, sequential recommender at 90–180s/call.
8	Latency (per-run wall-clock)	8	3	A	A: deterministic + fingerprint short-circuit. B: every page synchronous LLM; recommender tail dominates.
9	Maintainability	8	7	A	A: rubric-as-config + incremental migrations. B leaner but single-schema + <code>url LIKE</code> correctness hazard.
10	Developer experience	7	8	B	B's <code>--dry-run</code> , async ergonomics, OTEL, per-domain YAML = smoother first hour. A's CLI is broader/power-user.

#	Dimension	A	B	Win ner	One-line rationale
11	Testing support	9	3	A	A: 342 offline tests + CI, incl. tests for the new auditor/validator. B: 1 smoke test, no tests/ , no CI.
12	Ease of future upgrades	8	5	A	A has the v4 baseline wired + migrations. B missing site-level play; upgrades land on thin test scaffolding.
13	Monitoring and observability	8	8† (7)	Tie	A: queryable per-page journey (agent_traces + aeo trace). B: industry-standard OTLP (but inert with no collector).
14	Security considerations	7	4	A	A enforces robots.txt + rate limiting. B has neither; url LIKE %domain% weakens tenant isolation.
15	Production readiness	8	4	A	A: migrations, queue, Dockerfile, CI, 342 tests. B: no Dockerfile (compose starts only Postgres), no tests, unwired flagship.
16	Data management strategy	8	6	A	A: incremental migrations + 3 durability layers. B has the nicest immutability primitive (30-day lock) but no migrations, no site-level model.
17	Cost efficiency	8	4† (3)	A	A: 8/10 criteria zero-LLM + fingerprint skip. B: per-page inference; sequential recommender. (B credited for lean code TCO.)
18	LLM orchestration quality	7	6† (5)	A	A's LLM is wired + robust (3-strategy JSON, running auditor). B's design is more elegant but its two marquee features are unwired.
19	AEO optimization effectiveness	8	5	A	A: full v4 play incl. site-level missing-page diff + Perplexity test + feedback loop. B: per-page only, no missing-page concept.
20	Long-term sustainability	8	4	A	A: wired, tested, migratable surface. B: thin test net + single-schema + correctness bug offset its infra edge.
	Totals	156	106	A	A wins 17 · B wins 2 · 1 tie

Group subtotals

Theme (dimensions)	A	B
Architecture & Modularity (1–4)	30	25
Scale & Reliability (5–8)	32	17
Maintainability & DX (9–12)	32	23
Ops & Production (13–16)	31	22
Product & Cost (17–20)	31	19
Total	156	106

The only theme where B is competitive is Architecture & Modularity, and even there it loses on the two dimensions that test **delivered** (wired) capability rather than design intent.

PHASE 3 — Implementation Gap Analysis

3.1 Architecture A

Completed and production-ready (wired + tested):

- 10-criterion deterministic scoring engine, 8 of 10 zero-LLM (**scoring/scorers/__init__.py:36-47**).
- Versioned Reference Architecture Generator (L1+L2+L3) with content-hash reuse (**reference/blueprint.py:221-251**, **repos/blueprints.py:34-81**).

- **Site-level Coverage Diff** with thin-cluster detection (`processor/coverage_diff.py:166-178`) feeding net-new content briefs (`report/site_builder.py:97-100`).
- Independent validator + adversarial auditor + $\text{retry} \leq 3$, **all reachable from the run path** (`orchestrator.py:231-232` → `analysis.py:142-160`) and covered by `test_adversarial.py`, `test_independent_validator.py`, `test_validation.py`.
- Postgres job queue (`worker.py`, `repos/jobs.py:31-60`), incremental migrations 0001–0009, Error Sink page isolation, `agent_traces` + `aeo trace`, Docker/compose, CI, 342 offline tests.

Missing / incomplete / technical debt:

- **Sync psycpg2 inside an async orchestrator** — DB calls block the event loop per page (`storage/db.py:40-68`); the single biggest architectural wart.
- **Configs not pinned per run** (`framework.yaml`, `best_practices.yaml`): editing them between runs changes gap scores even though the blueprint version is pinned — a week-over-week comparability risk.
- Multi-topic taxonomy is **single-topic ("PEV") hardcoded** (`framework.yaml:19`); generalizing is config-only but not yet done.
- Only **6 of 10 criteria have simulation appliers**; schema/citation/load-speed/render are advisory-only in the re-score loop.
- Per-host **RateLimiter is per-process** — naive horizontal scale-out can exceed a domain's rate budget.
- Doc drift: "LangGraph" label vs. the actual custom async orchestrator; stale test counts; committed `.env` targets the wrong DB (`FIRECRAWL_API_KEY`, no `AEO__` * prefixes) and was deliberately left untouched because it holds a live secret.
- Unbounded JSONB `evidence` and `agent_traces` (no retention/partition policy).

Risks: single PostgreSQL is a SPOF for queue + data + traces; LLM cost is uncontrolled when validation is enabled site-wide (no per-run budget); Perplexity citation sends full URL+question to a third party (PII-leak surface); integration tests skip in CI so the first real migration is the first real deploy.

3.2 Architecture B

Completed and production-ready (wired + tested):

- Async-native crawler with SHA-256 change gate and tenacity retry (`agents/crawler.py`).
- Ollama blueprint generation with section-type normalization and JSON-repair (`agents/reference_generator.py`).
- Per-page coverage diff (Ollama semantic + deterministic entity/schema), fanned out via `asyncio.gather` (`agents/processor.py:110`).
- Deterministic recommendation gates (`agents/validator.py::validate_recommendations`).
- Frozen/versioned blueprint with a 30-day lock window — the single cleanest immutability primitive in either codebase (`models/blueprint.py:72-152`).
- Real OpenTelemetry OTLP tracing, uniform force-IPv4, per-domain YAML, `--dry-run` demo.

Missing / incomplete / technical debt:

- **No site-level / missing-page coverage diff** — the v4 headline. The blueprint has no ideal-sitemap concept; coverage is per-page only.
- **Adversarial auditor + 4-track engine-routed evaluator are built but never called** from `cli.py/processor.py` (verified) — dead code behind `smoke_test.py`.
- **No recommend → re-score → retry improvement loop** (B's "3x retry" is only the auditor's LLM circuit breaker).

- **No validated-wins feedback loop, no Perplexity real-world citation test** (the auditor only does a HEAD reachability check), **no per-page or site report artifact**.
- **No test suite** beyond one smoke test; **no CI**; **no Dockerfile** (compose starts only Postgres).
- ``get_content_hashes`` uses ``url LIKE %domain%`` (`db/queries.py:12-17`) — substring match that collides across domains.
- **Sequential recommender** loops Ollama calls at a documented 90–180s/call (`recommender.py:111-122`, `config.py:23`).
- Single idempotent `schema.sql` instead of incremental migrations; declared deps `google-genai` and `xxhash` unused.

Risks: every core stage depends on Ollama, so availability/quality of one local model gates the whole pipeline; LLM-judged coverage is non-deterministic (week-over-week scores can drift with no content change); no resumable run; the unwired flagship features mean the most-advertised capabilities deliver zero runtime value today.

PHASE 4 — Pros & Cons

Architecture A

Advantages

- Deterministic-first: reproducible, cheap, fast scores; LLM never blocks.
- Complete v4 implementation incl. the site-level missing-page play and a wired validation stack.
- Strong engineering hygiene: 342 offline tests, incremental migrations, CI, Error Sink, queue worker.
- Operationally scalable: Postgres-as-queue lets N stateless workers run with no broker.
- Auditable: full evidence trail per criterion + per-page trace journey.

Disadvantages / weaknesses

- Heavier surface (10k LOC) and a real simplicity tax in places.
- **Sync psycpg2 in an async context** (the standout architectural debt).
- Configs not pinned per run; thresholds partly hardcoded; single-topic taxonomy.
- Custom observability (queryable but non-standard) rather than OpenTelemetry.
- Single Postgres SPOF; uncontrolled LLM cost when validation runs site-wide.

Architecture B

Advantages

- Lean, readable, **async-native** end to end; clean `asyncio.gather` fan-outs.
- Genuinely **AI-native** design (engine-routed prompts, 4-track evaluator) — strong ideas.
- **Best-in-class operational ergonomics:** OpenTelemetry, per-domain YAML onboarding, `--dry-run` demo, uniform force-IPv4.
- Elegant data immutability (frozen blueprint + 30-day lock + content-hash).

Disadvantages / weaknesses

- **Two flagship features unwired** (auditor, engine-routed evaluator) — present but inert.
- **Missing the v4 site-level coverage play** entirely.

- **Effectively untested** (1 smoke test, no CI, no Dockerfile).
- **Model-bound performance/latency/cost**; sequential recommender bottleneck.
- Correctness/security hazards (`url LIKE %domain%`; no robots.txt/rate limiting).
- Non-deterministic coverage undermines week-over-week comparability.

PHASE 5 — Best-of-Both Analysis

Can the architectures be combined? — Yes, but asymmetrically.

This is **not** a case of two comparable rivals to be 50/50 merged. A is a near-complete, wired, tested v4 system; B is a lean, elegant, partially-wired prototype that is missing the v4 headline capability and a test suite. A symmetric merge would be expensive and would import B's largest liabilities (LLM-first non-determinism, untested surface). The correct pattern is **A as the base, with B as an idea donor**.

Take from Architecture B (port onto A) — high value, low risk

Port	Source in B	Why	Effort
OpenTelemetry OTLP export	<code>utils/observability.py</code>	Add standards-aligned distributed tracing *alongside* A's <code>agent_traces</code> table — best of both.	S
Per-domain YAML onboarding	<code>domain_config.py</code> , <code>domains/*.yaml</code>	Cleaner multi-tenant onboarding than A's single <code>framework.yaml</code> ; drives <code>engine_target</code> , seed queries, competitors per domain.	S–M
`--dry-run` in-memory demo mode	<code>cli.py:229-344</code>	Stakeholder demos + fast iteration with no DB; closes a real DX gap in A.	S
Per-engine prompt-emphasis routing	<code>coverage_diff.py:_ENGINE_EMPHASIS</code>	A carries <code>engine_target</code> but doesn't yet use it to shape prompts; B's emphasis pattern makes it influence output.	S
asyncpkg async-DB direction	<code>db/queries.py</code> (whole pattern)	B proves the asyncpkg path; a roadmap fix for A's sync-in-async wart (larger refactor, not immediate).	L (roadmap)

Take from Architecture A (the base) — keep verbatim

Deterministic-first scorer engine · site-level Coverage Diff + ideal sitemap · wired independent validator + adversarial auditor + `retry≤3` · validated-wins feedback · Postgres queue + incremental migrations · per-page + site reports · 342-test suite + CI.

Explicitly DO NOT take from B

- Its **LLM-first core** (forfeits A's determinism, reproducibility, cost, and latency wins).
- Its **single-schema-no-migrations** storage model.
- Its **untested surface** and unwired flagship code.

Integration challenges and merge risks

- **OTLP + `agent\_traces` duplication**: run both deliberately (queryable journey + portable spans); avoid double-instrumentation overhead by wrapping at the existing `trace_step` seam.
- **Config-source reconciliation**: B's per-domain YAML must feed A's `reference_architecture` settings without creating two sources of truth (and must be **pinned per run** to fix A's existing comparability gap at the same time).
- **Engine-emphasis must not break determinism**: keep prompt routing on the *LLM-refined* paths only; never let it perturb the deterministic scorers.

- People/ownership alignment (see §6): the B components worth porting are observability, onboarding, and infra — Sanjith's named ownership area — so the transplant maps cleanly onto existing responsibilities rather than cutting across them.
-

PHASE 6 — Final Recommended Architecture

Architecture A, hardened with five transplants from B. Full diagram:

[architecture_diagrams/03_final_recommended.md](#); request lifecycle: [04_request_lifecycle.md](#); deployment: [05_deployment.md](#).

Overview

Keep A's deterministic-first pipeline and complete v4 feature set as the system of record. Layer on B's operational ergonomics (OTLP, per-domain YAML, dry-run) and fold engine-routing into A's existing [engine_target](#) plumbing. Put the asyncpg migration on the roadmap to retire A's one structural wart.

Component list (final)

- **Onboarding:** per-domain YAML (from B) → drives A's reference-architecture config, pinned per run.
- **Reference generator:** A's L1+L2+L3, with B's per-engine prompt emphasis on the L3/gap LLM paths.
- **Crawler:** A's discovery + prioritization + Crawl4AI + content-hash gate + robots/rate-limiting.
- **Processor:** A's 12 extractors + 10 scorers (8 deterministic) + Dual-Layer Gap.
- **Site-level Coverage Diff:** A (ideal sitemap → missing/thin pages).
- **Recommender:** A's schema/entity/content generators.
- **Validation:** A's $\text{retry} \leq 3$ + independent validator (+Perplexity) + adversarial auditor (all wired).
- **Reporting:** A's per-page + site reports + validated-wins feedback.
- **Utilities:** A's async orchestrator + queue worker + Postgres; **dual observability** ([agent_traces](#) + OTLP from B); dry-run mode from B; weekly audit-cycle.

Data flow, service interactions, lifecycle

Unchanged from A's wired flow (Phase 1.1), with onboarding YAML at the front, engine-emphasis on LLM prompts, and OTLP spans emitted alongside trace rows. See the lifecycle sequence diagram ([04_request_lifecycle.md](#)).

Future scalability plan

- 1 **Now** — 1 VM, 1 worker, co-located Postgres.
- 2 **Scale-out** — N stateless workers claim from the Postgres queue (no broker).
- 3 **Managed DB** — move Postgres to a managed instance + read replicas for reporting.
- 4 **Async DB** — migrate psycopg2 → asyncpg (B's proven pattern) once event-loop blocking dominates the profile.
- 5 **Multi-region** — per-region workers, central blueprint store, for latency/residency.

Why this is the best choice

- **Technical:** keeps the only implementation that is complete, wired, and tested, and fixes A's worst wart on a roadmap while importing B's best operational ideas at low risk.

- **Product:** preserves the v4 differentiator (site-level "which pages should exist but don't") that B does not have, while adding B's engine-target product angle and demo-ability.
 - **Team productivity:** builds on the larger, tested codebase the team already iterates on; the transplanted pieces map onto existing ownership (Sanjith = infra/observability/onboarding; Kenneth = processor/validation).
 - **Long-term maintenance:** 342 tests + incremental migrations + config-as-code give a real safety net and evolution path; the asyncpg roadmap removes the structural debt without a rewrite.
-

PHASE 7 — Executive Summary

- 1 **Current state.** Two independent builds of the same v4 spec. A (Kenneth): deterministic-first, 10k LOC, 342 tests, feature-complete, some debt. B (Sanjith): LLM-first, 2.4k LOC, async-native, elegant but partially-wired and effectively untested.
 - 1 **Findings.** A scores **156/200** vs B **106/200** across 20 dimensions, after an adversarial fairness review. A wins 17 dimensions; B wins Pipeline Complexity and Developer Experience; Observability is a tie. B's two flagship features (adversarial auditor, engine-routed evaluator) are **built but not wired into its run path**, and B lacks the v4 **site-level missing-page** capability entirely.
 - 1 **Major risks.** *If B were chosen*: no tests/CI/Dockerfile, model-bound latency/cost, missing v4 headline, non-deterministic scores. *In A (to manage)*: sync-DB-in-async, single Postgres SPOF, configs not pinned per run, uncontrolled LLM cost, stale docs/.env.
 - 1 **Major opportunities.** Port B's OpenTelemetry, per-domain YAML onboarding, dry-run mode, and engine-prompt routing onto A; pin configs per run; put asyncpg on the roadmap.
 - 1 **Recommended direction.** **Adopt A as the system of record; transplant five B components. Do not merge codebases; do not rebuild on B.**
 - 1 **Expected benefits.** Keep a complete, tested, scalable, reproducible, low-cost pipeline; gain standards-aligned observability, cleaner onboarding, demo-ability, and a real engine-target product angle; remove A's structural DB debt over time.
 - 1 **Recommended next steps (90 days).** (a) Freeze A as baseline; (b) port per-domain YAML + pin configs per run; (c) port OTLP export alongside `agent_traces`; (d) port `--dry-run`; (e) wire engine-emphasis into LLM prompts; (f) reconcile `.env/docs` and run migrations `0001-0009` against a live Postgres in CI; (g) harvest B's blueprint-lock idea as a hardening reference.
 - 1 **Estimated migration effort. Low.** No data migration (A is the base). The five transplants are S/M each; the asyncpg refactor is the only L item and is roadmap, not blocking. Rough order: ~2–4 engineer-weeks for transplants + hardening, excluding asyncpg.
 - 1 **Estimated engineering effort remaining on A to "fully production-hardened." Moderate.** Pin configs per run (S), add per-run LLM budget guard (S), make RateLimiter cross-worker (M), add migration smoke test to CI (S), retention policy for traces/JSONB (S), redact Perplexity inputs (S), reconcile `.env/docs` (S), asyncpg migration (L, roadmap). Rough order: ~3–6 engineer-weeks excluding asyncpg.
-

8. Adversarial fairness review — what was challenged and adjusted

A dedicated reviewer audited the scoring for bias toward the larger codebase. Outcome: the result is directionally robust, but four B scores were raised (applied above; A scores were deliberately **not** lowered, with reasons):

- **Observability 7 → 8 (tie).** B's OpenTelemetry OTLP is the industry standard vs A's custom `agent_traces` table. Credited as a genuine tie (A retains the queryable per-page journey; B's spans are inert with no collector configured, default `OTEL_ENDPOINT=""`).
- **LLM orchestration 5 → 6.** B's 4-track engine-routed async design is genuinely more elegant; credited for design even though unwired. A stays 7 because A's LLM path is wired, robust (3-strategy JSON), and its auditor actually runs.
- **Scalability 4 → 5.** B can scale via stateless replicas and has real async fan-out. A stays 8 for the queue/worker primitive.
- **Cost efficiency 3 → 4.** B's lean code lowers *maintenance* TCO. A stays 8 because the dimension's dominant axis is *inference cost per page at scale*, where deterministic-first wins decisively.

The reviewer's most B-favorable honest reading lands B at ~110–115 (vs the 106 applied here, which conservatively declined the symmetric A-reductions). Either way, A wins by ~45–50 points. The fairness reviewer's own verdict: *"A is the clear production winner; B's true value was under-scored by ~10 points."* That correction is incorporated and does not change the recommendation.

FINAL DECISION FOR TOMORROW'S LEADERSHIP MEETING

Recommended architecture choice: Architecture A (Kenneth's `page_crawler`).

Merge or not: Do not merge the two codebases. Adopt A as the system of record and **port five specific components from B** (OpenTelemetry OTLP, per-domain YAML onboarding, `--dry-run` demo mode, per-engine prompt-emphasis routing, and the asyncpg async-DB direction as a roadmap item). B remains valuable as an idea donor, not a base.

Exact reasoning:

- A is the **only complete v4 implementation**: it has the site-level missing-page Coverage Diff, a wired independent validator + adversarial auditor + retry loop, and a validated-wins feedback loop. B is missing the headline site-level play, and its two flagship features are **built but not wired** (verified in code).
- A is **measurably more production-ready**: 342 offline tests + CI vs **one** smoke test and no CI; incremental migrations vs a single schema file; a Dockerfile + queue worker vs none.
- A is **cheaper, faster, and reproducible** by design (deterministic-first; 8/10 criteria need no LLM), whereas B is model-bound with a sequential recommender at 90–180s/call.
- A wins **17 of 20** dimensions (156 vs 106) even after an adversarial fairness review that deliberately favored B.

Expected benefits: a complete, tested, scalable, low-cost, reproducible AEO engine now; plus B's best operational ideas (standards-aligned tracing, clean onboarding, demo-ability, engine-target routing) at low integration risk; with A's one structural wart (sync DB) on a clear roadmap.

Risks (and mitigations): A's sync-DB-in-async (→ asyncpg roadmap), single Postgres SPOF (→ managed DB + replicas), configs not pinned per run (→ pin during the YAML port), uncontrolled LLM cost (→ per-run budget guard). None are blocking; all are scoped in \$7.9.

Immediate next engineering actions: (1) freeze A as baseline; (2) port per-domain YAML onboarding and pin configs per run; (3) add OTLP export beside `agent_traces`; (4) port `--dry-run`; (5) wire engine-emphasis into the LLM prompts; (6) reconcile `.env/docs` and exercise migrations in CI against a live Postgres.

***Bottom line:** Architecture A is the foundation. Architecture B is the parts bin. Ship A; harvest B.*